FIAP / GLOBAL SOLUTION 2026.2

Relatório Técnico — Automação Inteligente de Triagem de Obstáculos Urbanos

ROTA LIVRE

Victoria Franceschini Pizza · RM 550609

Robotic Process Automation

12 de novembro de 2026

Sumário

1. Introdução e objetivo	3
2. Processo e referência ao PDD	3
3. Arquitetura, tecnologias e pastas	3
4. Funcionamento, regras e exceções	4
5. Logs e relatório JSON	5
6. Integração com FastAPI e o MVP	5
7. Metodologia, testes e evidências	6
8. Segurança, LGPD e responsabilidades	7
9. Limitações e conclusão	8
10. Referências	8

1. Introdução e objetivo

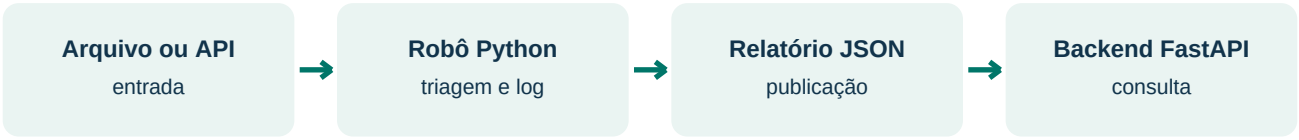
O ROTA LIVRE mapeia barreiras de acessibilidade em calçadas, rampas e travessias. Relatos incompletos, coordenadas fora da faixa geográfica e repetições prejudicam a qualidade do lote. O robô padroniza a conferência desses dados, sinaliza rejeições e atribui prioridades preliminares para apoiar a triagem. A classificação é uma decisão operacional inicial, sujeita a revisão humana e vistoria.

2. Processo e referência ao PDD

O PDD PDD-RL-RPA-001, versão 1.0 de 12/11/2026, define o processo-alvo de coleta, validação, deduplicação, classificação e publicação. A cópia consultada foi localizada no projeto anterior ROTA_LIVRE_RPA_GitHub_COMPLETO. A implementação atual usa um relatório em arquivo para a publicação. Deduplicação por proximidade, revisão humana e controle de idempotência persistente entre execuções permanecem como evolução. Cada reexecução gera um novo relatório com um identificador próprio.

3. Arquitetura, tecnologias e pastas

O processamento usa Python e sua biblioteca padrão, com argparse para a CLI, urllib para HTTP, json para serialização, logging para auditoria e unittest para testes. FastAPI e Uvicorn servem a consulta dos resultados. React e Leaflet compõem o mapa existente do MVP.



Local	Responsabilidade
main.py e config.py	Entrada da CLI, parâmetros, ciclo de execução e códigos de saída.
src/	Leitura, validação, deduplicação, classificação, publicação, logs e integração.
dados/ e logs/	Lote de exemplo, relatórios locais e arquivos operacionais com rotação.
integracao_backend/	Roteador, aplicação de exemplo e override Docker Compose.
tests/, evidencias/ e docs/	Regressão, resultados observados e documentação técnica.

4. Funcionamento, regras e exceções

O leitor recebe JSON UTF-8 local ou consulta uma API GET. A entrada deve conter uma lista diretamente ou sob uma chave configurada no modo API. O laço for de src/robo.py percorre o lote com posições iniciadas em 1. A condição if erros registra a rejeição e avança com continue. Os campos exigidos são id, tipo, descricao, latitude, longitude e status. O ID aceita texto não vazio ou inteiro. Latitude e longitude são numéricas e respeitam, respectivamente, os intervalos de -90 a 90 e -180 a 180.

O identificador de comparação é convertido em texto e tem seus espaços externos removidos. A primeira ocorrência válida é classificada. A condição if chave in primeira_posicao marca as seguintes como duplicadas e registra o motivo e a posição original. Por exemplo, 101 e “ 101 ” são equivalentes. Os registros originais são preservados.

Decisão em src/classificador.py	Prioridade
if — rampa bloqueada ou travessia perigosa	Alta
elif — calçada danificada	Média
else — demais tipos válidos	Baixa

A classificação normaliza maiúsculas, acentos e espaços. O leitor trata arquivo ausente ou vazio, erro de sintaxe e estrutura incompatível. A consulta HTTP usa timeout e tentativas limitadas. Falhas de conexão, respostas vazias ou inválidas e HTTP 408, 429 e 5xx podem ser reavaliados. Contingência local exige configuração explícita e identifica a origem no relatório. Falha definitiva retorna código 1. Um lote legível com rejeições pode gerar relatório com código 0 e situação sucesso_parcial.

5. Logs e relatório JSON

Cada execução recebe um UUID. O log registra início, término, data e hora UTC, origem, totais, erros e situação. Advertências incluem a posição e o motivo. A falha da API registra tentativas e erro final. O log ativo gira em 1 MiB e mantém até três cópias. A saída JSON é gravada em arquivo temporário e publicada por renomeação na mesma pasta, preservando a entrada.

O relatório inclui execucao_id, processado_em, origem, situacao, resumo, validos, invalidos e duplicados. O resumo registra total, validos, invalidos e duplicados. A saída HTTP inclui metadados, resumo, totais por prioridade e uma projeção dos válidos com ID, tipo, coordenadas, prioridade e status_triagem. Os campos de descrição e motivos de rejeição permanecem no relatório interno.

6. Integração com FastAPI e o MVP

O backend real contém GET /api/obstacles, que retorna total e a lista obstaculos. A rota /obstaculos indicada no PDD retornou 404 no teste local. A origem real fornece ID inteiro, tipo, coordenadas e status, sem descricao. O adaptador explícito --adaptador-mvp preenche somente a ausência desse campo com uma indicação de indisponibilidade. O contrato da rota original foi preservado.

O backend registra o roteador quando RPA_RELATORIOS_DIR é definido. O consumidor seleciona o relatório compatível mais recente e valida metadados, contagens e registros. GET /rpa/obstaculos-processados retorna a triagem. GET /rpa/resumo retorna os metadados e totais. Ambos retornaram HTTP 200 com relatório válido, 404 sem relatório e 503 quando havia apenas um relatório corrompido. Os testes usaram o backend real em loopback. O proxy Vite manteve o contrato de /api/obstacles e o frontend passou em build e lint. O mapa continua consultando a rota original.

O override Compose prevê montar os módulos Python e o diretório de relatórios em modo somente leitura. A configuração foi validada com docker compose config. A execução integrada em contêiner e a publicação dos endpoints em produção permanecem fora da validação realizada.

7. Metodologia, testes e evidências

Os testes usam subprocessos, diretórios temporários, hashes de arquivos e requisições HTTP em loopback. O lote de exemplo produziu seis recebidos, quatro válidos, um inválido e um duplicado. As prioridades foram alta, alta, média e baixa. O backend real forneceu dois registros, ambos válidos, com uma prioridade alta e uma média. Os totais coincidiram no log, no relatório e na resposta HTTP.

Teste automatizado do RPA	Resultado observado
API indisponível	Limite de tentativas, log de erro e saída controlada.
Execuções limpas	IDs distintos e situação sucesso.
IDs equivalentes	Duplicado identificado, entrada preservada.
JSON malformado	Erro registrado, nenhum relatório gerado.
Latitude inválida	Registro rejeitado, processamento do lote mantido.
Leitor e endpoints	HTTP 200, 404 e 503 com respostas esperadas.
Lote e logs	Seis posições processadas, três prioridades e totais consistentes.
Rotação	Log ativo e três cópias.

Oito testes do RPA e três testes de métricas do backend passaram na Etapa 4. As evidências 07 a 11 registram a etapa anterior, incluindo o log normal, as falhas e o JSON do exemplo. As evidências 12, 13 e 14 registram a execução no backend real em loopback, os corpos das respostas e as regressões.

8. Segurança, LGPD e responsabilidades

A finalidade da triagem é organizar dados sobre barreiras urbanas e apoiar sua localização. A projeção HTTP limita os campos ao uso operacional. O log evita descrições, IDs de obstáculos, corpo e URL da origem. O relatório interno retém os registros do lote, inclusive campos livres que podem conter dados pessoais. O Git ignora relatórios operacionais, arquivos de log e ambientes locais. O leitor valida a estrutura e limita a leitura ao diretório configurado.

Na operação, a equipe deve restringir escrita ao robô e conceder somente leitura ao backend, revisar quais IDs e coordenadas podem ser expostos e estabelecer prazos de retenção e descarte para relatórios e backups. A rotação controla espaço em disco e precisa ser complementada por política de retenção. Antes de expor a integração externamente, o responsável pelo serviço deve aplicar autenticação, autorização e HTTPS, revisar a finalidade e a base legal cabível e definir procedimentos para atendimento dos titulares e incidentes, conforme a Lei nº 13.709/2018. Essas medidas operacionais requerem decisão e configuração do controlador.

9. Limitações e conclusão

A triagem está funcional e foi validada com o backend do repositório em execução local. O mapa preservou seu contrato de consulta. A exibição de prioridades no mapa, a validação do override em contêiner, a habilitação em produção e a política operacional de retenção são trabalhos futuros. A classificação é preliminar. Os dados de origem verificados correspondem aos dois registros fornecidos pela rota implementada no MVP.

Link da demonstração audiovisual a acrescentar na versão final.

10. Referências

PIZZA, Victoria Franceschini. PDD-RL-RPA-001. ROTA LIVRE, versão 1.0, FIAP, 2026.

ROTA LIVRE. Código, README e evidências do RPAGS.
https://github.com/victoriafpizza/GlobalSolution2026/tree/main/RPAGS

FASTAPI. Bigger Applications — Multiple Files.
https://fastapi.tiangolo.com/tutorial/bigger-applications/

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais.
https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm